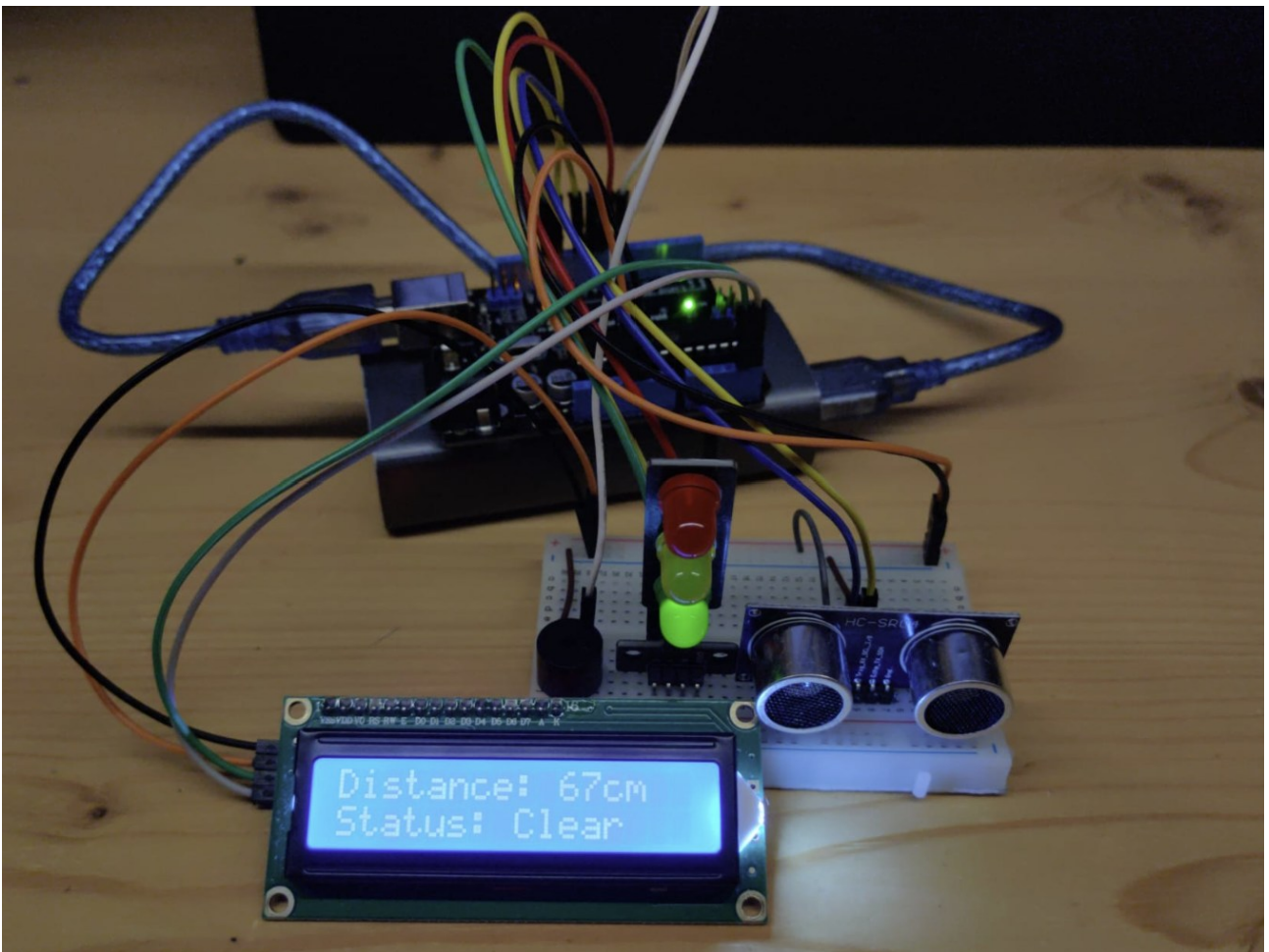


Ultrasonic Distance System with Real-Time Data Visualization

Criar, atualizar e personalizar um sumário

This smart parking assistant utilizes an ultrasonic sensor and a moving average filter to provide stable distance tracking via an I2C LCD, LED traffic lights, and a reactive buzzer. Optimized with non-blocking code for zero-lag responsiveness and MATLAB integration, the project demonstrates essential principles of signal processing and efficient timing for embedded systems.



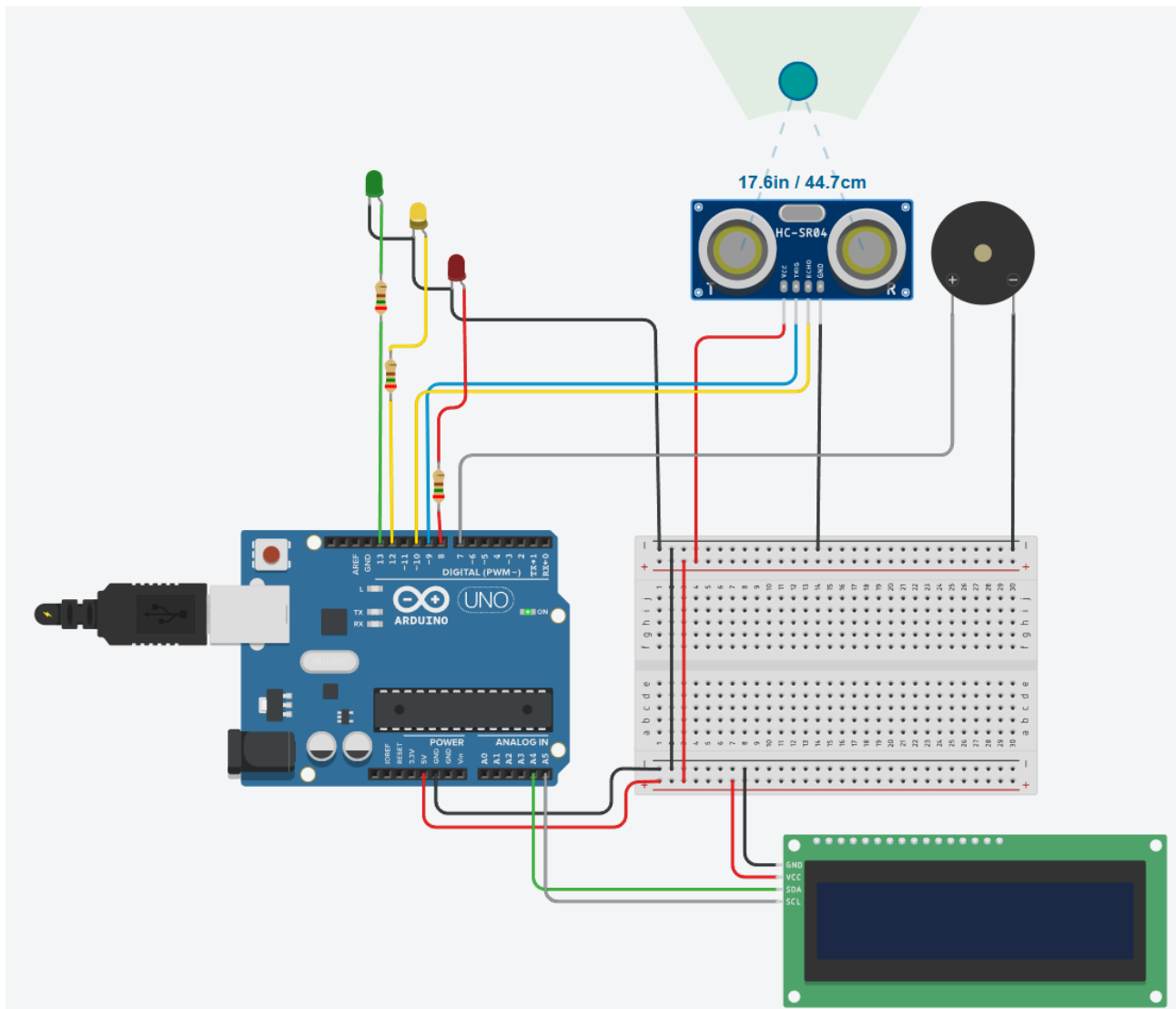
Components and supplies

- 1 16x2 LCD display with I²C interface
- 1 40 colored male-female jumper wires
- 1 Arduino Uno Rev31 Ultrasonic Sensor - HC-SR04
- 1 5V Mini Traffic Light Red Yellow Green 5mm LED Display Module

- 1 Battery 5V (PowerBank)
- 1Piezo buzzer

Project description

This project focuses on building a highly responsive distance-monitoring system, similar to a vehicle's parking assistant. By utilizing an HC-SR04 ultrasonic sensor, the system provides multi-layered feedback via an I2C LCD, a three-stage LED traffic light, and a variable-frequency buzzer. Unlike basic implementations, this system is optimized for stability through data smoothing and a non-blocking code structure.



Functional Implementation

The system categorizes distance into three distinct safety zones to guide the user:

Safe Zone (> 50cm): Green LED is active; the display indicates a "Clear" status.

Warning Zone (20cm - 50cm): Yellow LED is active; the buzzer pulses with a frequency that increases as the object approaches.

Danger Zone (< 20cm): Red LED is active; the buzzer frequency increases significantly, transitioning to a continuous tone when the distance falls below 5cm.

Technical Optimizations

To ensure great performance, the following software techniques were implemented:

Moving Average Filter: To prevent erratic readings and "jitter" on the LCD, the software maintains a rolling array of the last five measurements. It calculates the average of these values, resulting in a stable and reliable output.

Non-Blocking Timing (millis()): By replacing the standard `delay()` function with `millis()`-based logic, the microcontroller can update the sensor data and the LCD while simultaneously controlling the buzzer's pulse rhythm. This prevents the system from "freezing" during acoustic alerts.

Sensor Timeout Management: The `pulseIn()` function is configured with a 25ms timeout. This ensures the loop continues to run immediately even if the sensor fails to receive an echo, maintaining a high system refresh rate.

Data Streaming: The system is pre-configured to output raw distance values to the Serial port in a format compatible with MATLAB and Octave for real-time graphing and signal analysis.

Future Outlook & Enhancements

IoT & Wireless Connectivity: Integrate Bluetooth (HC-05) or WiFi (ESP32) modules to stream real-time distance data to a mobile application dashboard.

360° Surround Monitoring: Expand the hardware to a multi-sensor array to eliminate blind spots and provide full-perimeter coverage around a vehicle.

Advanced Signal Processing: Implement a Kalman filter to significantly reduce sensor noise and improve tracking precision for fast-moving objects.

Code

```
// // 1. PIN DEFINITIONS & LIBRARIES
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

```

// Initialize the LCD at address 0x27 with 16 columns and
2 rows
// Note: If 0x27 doesn't work, try 0x3F
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Ultrasonic sensor pins
const int trigPin = 9;
const int echoPin = 10;

// Traffic light LED pins
const int ampelR = 8; // Red LED
const int ampelY = 12; // Yellow LED
const int ampelG = 13; // Green LED

// Buzzer pin
const int buz = 7;

// // 2. VARIABLES FOR DISTANCE & MOVING AVERAGE
long duration = 0;
int distance = 0;

const int numReadings = 5;
int readings[numReadings];
int readIndex = 0;
long total = 0;
int averageDistance = 0;

// // 3. TIME CONTROL VARIABLES (Global)
unsigned long previousMillis = 0; // For buzzer timing
unsigned long lastLCDUpdate = 0; // For LCD refresh
timing
const int lcdInterval = 200; // Update LCD every
200ms
bool buzzerState = LOW;

```

```

// // 4. SETUP
void setup() {
    Serial.begin(9600);

    // LCD Initialization
    lcd.init();
    lcd.backlight();
    lcd.setCursor(0, 0);
    lcd.print("System Ready");
    delay(1000);
    lcd.clear();

    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    pinMode(ampelR, OUTPUT);
    pinMode(ampelY, OUTPUT);
    pinMode(ampelG, OUTPUT);
    pinMode(buz, OUTPUT);

    for (int i = 0; i < numReadings; i++) {
        readings[i] = 0;
    }
}

// // 5. MAIN LOOP
void loop() {
    unsigned long currentMillis = millis();

    // // 6. SENSOR MEASUREMENT
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);

```

```

digitalWrite(trigPin, LOW);

duration = pulseIn(echoPin, HIGH, 25000);
distance = (duration * 0.034) / 2;

if (distance <= 0) {
    distance = 200;
}

// // 7. CALCULATE MOVING AVERAGE
total = total - readings[readIndex];
readings[readIndex] = distance;
total = total + readings[readIndex];
readIndex = readIndex + 1;

if (readIndex >= numReadings) {
    readIndex = 0;
}
averageDistance = total / numReadings;

// // 8. TRAFFIC LIGHT LOGIC
if (averageDistance <= 20 && averageDistance > 0) {
    digitalWrite(ampelR, HIGH); digitalWrite(ampelY, LOW);
digitalWrite(ampelG, LOW);
}
else if (averageDistance > 20 && averageDistance <= 50)
{
    digitalWrite(ampelR, LOW); digitalWrite(ampelY, HIGH);
digitalWrite(ampelG, LOW);
}
else {
    digitalWrite(ampelR, LOW); digitalWrite(ampelY, LOW);
digitalWrite(ampelG, HIGH);
}

```

```

// // 9. BUZZER LOGIC
int pauseInterval = 0;
const int beepDuration = 10;

if (averageDistance <= 5 && averageDistance > 0)
pauseInterval = 1;
else if (averageDistance <= 10) pauseInterval = 100;
else if (averageDistance <= 15) pauseInterval = 300;
else if (averageDistance <= 30) pauseInterval = 700;
else if (averageDistance <= 50) pauseInterval = 1200;
else pauseInterval = 0;

if (pauseInterval == 1) {
    digitalWrite(buz, HIGH);
}
else if (pauseInterval == 0) {
    digitalWrite(buz, LOW);
    buzzerState = LOW;
}
else {
    if (buzzerState == HIGH) {
        if (currentMillis - previousMillis >= (unsigned
long)beepDuration) {
            digitalWrite(buz, LOW);
            buzzerState = LOW;
            previousMillis = currentMillis;
        }
    }
    else {
        if (currentMillis - previousMillis >= (unsigned
long)pauseInterval) {
            digitalWrite(buz, HIGH);
            buzzerState = HIGH;
        }
    }
}

```

```

        previousMillis = currentMillis;
    }
}

// // 10. LCD DISPLAY LOGIC
// We update the LCD only every 200ms to avoid
flickering
if (currentMillis - lastLCDUpdate >= lcdInterval) {
    lastLCDUpdate = currentMillis;

    lcd.setCursor(0, 0);
    lcd.print("Distance:      "); // Spaces clear old
digits
    lcd.setCursor(10, 0);
    lcd.print(averageDistance);
    lcd.print("cm");

    lcd.setCursor(0, 1);
    if(averageDistance <= 20) lcd.print("Status:
STOP!  ");
    else if(averageDistance <= 50) lcd.print("Status:
Caution ");
    else lcd.print("Status: Clear  ");
}

// // 11. OUTPUT FOR MATLAB
Serial.println(averageDistance);

delay(30);
}

```

Reference

<https://projecthub.arduino.cc/falzinger/ultrasonic-distance-system-with-real-time-data-visualization-811a7b>